

Az os modul szerepe

Hogyan találja meg a program a fájlokat, hogyan dolgozunk mappákkal, és mire jó az os.path?

import os

os.getcwd()

os.path.join()

```
import os
```

```
print(os.getcwd())  
print(os.listdir())
```



Mire való az os modul?

Az os modulon keresztül a Python közvetlenül tud dolgozni a gép fájlrendszerével.



- ▶ Ugyanazokat az információkat kérjük le, amelyeket kézzel is látsz a fájlkezelőben vagy a terminálban.
- ▶ A cél az, hogy a program ne fix helyekre támaszkodjon, hanem felismerje és kezelje a saját környezetét.

Lényeg: ne beégetett útvonalakkal dolgozzunk, hanem kérdezzük le és építsük fel őket tudatosan.

Mire fogjuk használni?



Megnézzük, honnan fut a program

Ez dönti el, hogy a relatív útvonalak honnan indulnak.



Bejárjuk egy mappa tartalmát

Meg tudjuk nézni, milyen fájlokkal és almappákkal dolgozhatunk.



Útvonalakat rakunk össze biztonságosan

Nem kézzel ragasztgatjuk a perjeleket, hanem erre való függvényeket használunk.



Művelet előtt ellenőrzünk

Létezik az útvonal? Fájl vagy mappa? Biztosan ezzel akarunk dolgozni?

Aktuális munkakönyvtár: CWD

CWD = Current Working Directory, vagyis a program aktuális kiindulási mappája.

Ha relatív útvonalat adunk meg, a Python ehhez a mappához képest keresi a fájlt.

```
import os

print(os.getcwd())
# például: C:\Users\Martin\projektek\demo
```

```
os.chdir("data")
print(os.getcwd())
# innentől a data mappa az aktuális hely
```

Fontos: a chdir átállítja a program nézőpontját. Ettől kezdve minden relatív útvonal máshonnan indul.

```
...
$ pwd
/home/martin/projekt

$ tree
.
├── data
├── src
├── test
└── ...

$ pwd
/home/martin/projekt

$ python main.py
/home/martin/projekt
```

► **Jó ellenőrzés: futtasd ugyanazt a programot másik mappából, és figyeld meg, mit ír ki a `getcwd()`.**

Abszolút és relatív útvonalak

Abszolút útvonal

Teljes cím: a meghajtótól vagy a gyökérkönyvtártól indul, ezért önmagában is egyértelmű.

```
C:\Users\Martin\projekt\data\input.txt  
/home/martin/projekt/data/input.txt
```

Relatív útvonal

Az aktuális munkakönyvtárhoz képest értelmezzük. Kényelmesebb, de tudni kell, honnan fut a program.

```
data/input.txt  
../kepek/logo.png  
./eredmeny/output.csv
```

A relatív útvonal nem rossz megoldás, csak mindig tudnunk kell, mihez képest relatív.

Jó minta: útvonal összeállítása `os.path.join()`-nal, utána ellenőrzés, és csak ezután a művelet.

Útvonalak Windows, Linux és macOS alatt

Gyakori hiba, amikor az útvonalat sima szöveggént ragasztjuk össze.

```
# kerülendő  
path = folder + "/" + filename  
path = "C:/Users/Martin/" + filename
```

Miért lehet ebből gond?

- ▶ Windows alatt gyakori a backslash: \
- ▶ Linux és macOS alatt a slash: /
- ▶ A \n és \t escape karakterek miatt könnyen hibás útvonal jöhet létre.

```
# jó minta  
import os
```

```
path = os.path.join(folder, filename)  
print(path)
```

Az `os.path.join()` az adott rendszerhez illő elválasztót használja, ezért a kód könnyebben működik más gépen is.

...

```
Windows: data\input.txt  
Linux:   data/input.txt
```

Fájlok és mappák listázása

`os.listdir(path)`

Visszaadja a mappa tartalmát, de csak a neveket kapjuk meg, nem a teljes útvonalakat.

```
import os

for nev in os.listdir("data"):
    print(nev)
```

Kérdés: mi történik, ha a data mappa nem létezik?

`os.scandir(path)`

Kényelmesebb, mert minden elemről külön meg tudjuk kérdezni, hogy fájl-e vagy mappa.

```
with os.scandir("data") as elemek:
    for e in elemek:
        if e.is_file():
            print("fájl:", e.name)
```

Válasz: hiba lesz, ezért érdemes előtte ellenőrizni, hogy az útvonal létezik-e.

Mappák létrehozása és kezelése



```
import os
```

```
os.mkdir("eredmeny")  
# csak egy mappaszintet hoz létre
```

```
os.makedirs("eredmeny/kepek/2026", exist_ok=True)  
# több szintet is létrehoz  
# exist_ok=True: nem gond, ha már létezik
```

```
os.rmdir("ures_mappa")  
# csak üres mappát töröl
```

Mire figyelj?

- ▶ A **mkdir** hibát dobhat, ha a mappa már létezik.
- ▶ **Többszintű mappákhoz** hasznos a **makedirs()**.
- ▶ **Törlés előtt** mindig írasd ki, pontosan mire mutat az útvonal.
- ▶ Az **rmdir** csak üres mappát tud törölni.

Mappáknál is ugyanaz a sorrend: útvonal
összeállítása → ellenőrzés → művelet.

Fájlműveletek: átnevezés, mozgatás, törlés

Az `os` modul főleg a fájlok helyével és nevével dolgozik. A fájl tartalmát továbbra is az `open()`-nel olvassuk vagy írjuk.

```
os.rename("regi.txt", "uj.txt")  
# átnevezés, másik útvonal esetén mozgatás is
```

```
os.replace("uj.txt", "archive/uj.txt")  
# cél fájl esetén felülírhat, ezért csak tudatosan
```

```
os.remove("temp.txt")  
# fájlt töröl, mappát nem
```

Művelet előtt érdemes végiggondolni:

- ▶ Létezik az útvonal?
- ▶ Fájlra vagy mappára mutat?
- ▶ **Biztosan ezt akarom módosítani vagy törölni?**
- ▶ **Kell róla mentés, mielőtt hozzányúlok?**

Az os.path legfontosabb függvényei



join()

útvonal összeállítása

```
os.path.join("data", "a.txt")
```

exists()

létezik-e az útvonal?

```
os.path.exists(path)
```

isfile()

fájlra mutat?

```
os.path.isfile(path)
```

isdir()

mappára mutat?

```
os.path.isdir(path)
```

abspath()

teljes útvonal készítése

```
os.path.abspath("data")
```

splitext()

név és kiterjesztés szétválasztása

```
os.path.splitext("kep.png")
```

Ezekkel már el lehet készíteni egy egyszerű, jól ellenőrzött fájlkezelő programot.

Útvonal szétszedése: basename, dirname, split



```
import os

path = "data/images/logo.png"

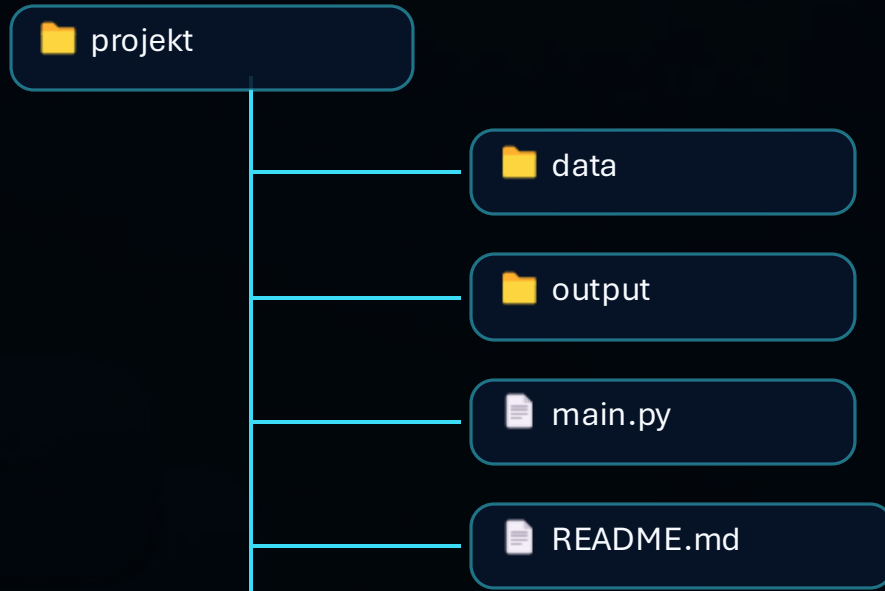
print(os.path.basename(path)) # logo.png
print(os.path.dirname(path)) # data/images
print(os.path.split(path)) # (data/images, logo.png)
print(os.path.splitext(path)) # (data/images/logo, .png)
```

Mikor hasznos?

- ▶ Amikor csak a fájlnevet akarjuk kiírni a teljes útvonal nélkül.
- ▶ Amikor kiterjesztés alapján szeretnénk szűrni.
- ▶ Amikor ugyanabba a mappába szeretnénk új fájlnevet készíteni.

Példa: a .txt fájlokat feldolgozzuk, a képeket viszont kihagyjuk.

Mintapélda: rendezett projektmappa



```
BASE_DIR = os.getcwd()
DATA_DIR = os.path.join(BASE_DIR, "data")
OUT_DIR = os.path.join(BASE_DIR, "output")

os.makedirs(OUT_DIR, exist_ok=True)
```

► **A program elején érdemes egyszer összerakni a fontos mappák útvonalát.**

► **Így később nem szóródnak szét a kézzel beírt útvonalak a kódban.**

► **Könnyebb átírni, tesztelni és hibát keresni.**

Biztonságos fájlkezelési minta

```
path = os.path.join("data", "input.txt")

if os.path.exists(path) and os.path.isfile(path):
    with open(path, "r", encoding="utf-8") as f:
        tartalom = f.read()
else:
    print("Hiányzó vagy hibás fájl:", path)
```

Miért biztonságosabb így?

- ▶ Nem feltételezzük vakon, hogy a fájl létezik.
- ▶ Nem próbálunk meg mappát fájlként megnyitni.
- ▶ Érthető hibaüzenetet adunk, ha valami hiányzik.
- ▶ Az encoding megadása segít elkerülni az ékezetes karakterekkel kapcsolatos hibákat.

Gyakorlati sorrend: útvonal összeállítása → ellenőrzés → művelet.

Mini demó: csak a .txt fájlok listázása



```
import os

mappa = "data"

for nev in os.listdir(mappa):
    ut = os.path.join(mappa, nev)
    torzs, kiterjesztes = os.path.splitext(nev)

    if os.path.isfile(ut) and kiterjesztes == ".txt":
        print("TXT fájl:", nev)
```

Mit gyakorolunk ezzel?

- ▶ **listdir** → a mappa tartalmának bejárása
- ▶ **join** → teljesebb útvonal készítése
- ▶ **isfile** → mappák kiszűrése
- ▶ **splitext** → szűrés kiterjesztés alapján

Gyakorlaton ebből könnyen tovább lehet lépni: darabszám, méret, átnevezés, másolás.

Tipikus hibák és gyors javítások

FileNotFoundError

Általában rossz munkakönyvtár vagy elért relatív útvonal okozza.

Írasd ki: `os.getcwd()`, majd ellenőrizd: `os.path.exists(path)`

PermissionError

Nincs jogosultság, vagy a fájlt éppen használja egy másik program.

Ellenőrzés: jó mappa, megfelelő jogosultság, nincs-e nyitva a fájl

Kézzel összefűzött út

Eltérő útvonalelválasztók vagy escape karakterek okozhatják.

`os.path.join()`

Véletlen törlés

Rossz mappában járunk, vagy túl általános útvonalat adtunk meg.

`exists + isfile/isdir + az útvonal kiíratása művelet előtt`

Hibakeresési rutin: írasd ki a CWD-t → írasd ki az útvonalat → `exists/isfile/isdir` → csak utána jöhet a művelet.

Kitekintés: os.path mellett ott van a pathlib is

Most az os és az os.path alapjait nézzük, de érdemes tudni, hogy modern Python-kódokban gyakran találkozunk pathlib-bal is.

```
# os.path stílus  
path = os.path.join("data", "input.txt")  
print(os.path.exists(path))
```

```
# pathlib stílus  
from pathlib import Path  
  
path = Path("data") / "input.txt"  
print(path.exists())
```

Lényeg: először értsük meg az útvonal és a CWD logikáját. Utána az os.path és a pathlib is sokkal könnyebben tanulható.